

Numerix

An Algorithmic Scientific Calculator Engine

Project Report

Student Name:	Biraja prasad mishra
Registration No.:	26BAC10012
Course Name:	Introduction to problem solving and programming
Course Code:	CSE-1021
Submitted To:	venkat prasad pathy
Submission Date:	28/09/2026

)

1. Introduction

For this project, I built a simple calculator program in Python called **Numerix**. The idea was to combine basic arithmetic with a bit of trigonometry and factorial calculation, all inside one menu-driven console application. Since this course covers programming fundamentals like loops, functions, conditionals and recursion, I wanted a project that actually uses all of these together in one working program, instead of writing separate tiny programs for each topic and never connecting them.

Numerix doesn't use any external math library on purpose — every operation, from a simple addition to a trigonometric value, is worked out using plain Python logic. This was a deliberate choice so that the internal working of each type of calculation stays visible instead of being hidden behind a library function call.

2. Problem Statement

Most people just use their phone's calculator app or type sums into a search engine, which is fine for everyday use but doesn't really show *how* a calculator works behind the scenes. As a student, I wanted to understand how basic mathematical operations — arithmetic, trigonometry and factorial — could be handled step by step using core programming logic, without relying on any built-in math library to do the actual work.

So the problem I set out to solve was: **build a menu-driven scientific calculator in Python that can perform basic arithmetic, return trigonometric values for standard angles, and calculate factorials — using only fundamental programming concepts (loops, conditionals, functions and recursion) rather than external libraries.**

3. Functional Requirements

The project is built around three major functional modules, matching the three categories shown in the main menu:

- **Arithmetic Module** — performs addition, subtraction, multiplication, division, floor division and modulus on two numbers entered by the user.
- **Trigonometry Module** — returns the sin, cos or tan value for one of five standard angles (0°, 30°, 45°, 60°, 90°), returned as an exact expression rather than a decimal.
- **Factorial Module** — calculates the factorial of a number entered by the user, using a recursive function.

Input / Output Structure

Every module follows the same basic pattern: the program asks the user for the operator/category first, then asks for whatever numbers or angle it needs, does the calculation, and prints the result back to the console. If something goes wrong (like dividing by zero or picking an angle that isn't supported), it prints a clear message instead of crashing.

Workflow

The program runs inside a loop. Every time it starts a cycle it asks the user if they want to perform a calculation. If the answer is "yes", it shows the menu, takes the operator and inputs, prints the result, and then loops back to ask again. If the answer is anything other than "yes", it prints a thank-you message and the program ends.

4. Non-Functional Requirements

- **Usability** — the menu is written in plain text with clear prompts, so anyone can use it without needing instructions.
- **Reliability** — the program doesn't crash on common bad inputs like division by zero or an unsupported trig angle; it shows a message and continues instead.
- **Maintainability** — related operations are grouped together with if-elif chains and a separate `fact ( )` function, so it's reasonably easy to find and edit a specific operation.
- **Error Handling Strategy** — invalid operators, invalid angles and division by zero are all caught with an if/else check and a friendly message rather than letting Python throw a raw exception.
- **Resource Efficiency** — since all the calculations are simple (no loops over large datasets, no heavy computation), the program runs instantly and uses almost no memory.

I didn't focus much on things like scalability or logging/monitoring since this is a small, single-user console tool rather than a system that needs to handle many users or be monitored in production — those requirements didn't really apply at this scale.

5. System Architecture

Numerix follows a simple, single-file, procedural architecture — there's no client-server split or database involved. The user interacts directly with the terminal, the main loop reads their input, decides which module to hand it off to, and prints the result straight back to the same terminal. The diagram below shows this at a high level.

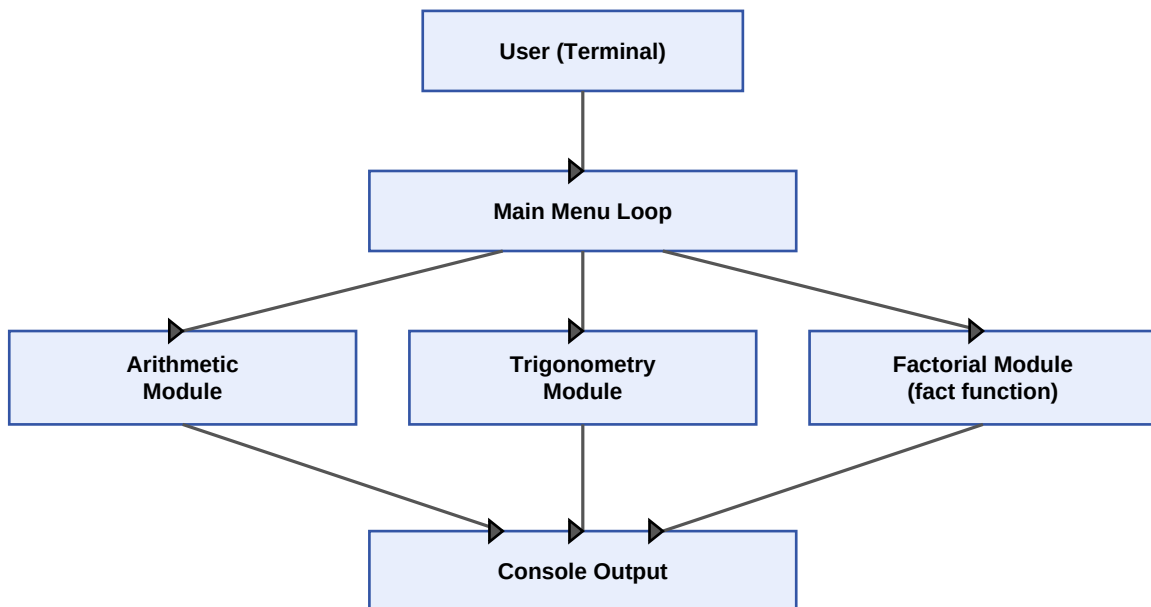


Figure 1: System Architecture Diagram

6. Design Diagrams

6.1 Use Case Diagram

Since Numerix only has one type of user, the use case diagram is fairly simple — the user can choose to perform any of the three calculation types, or exit the program.

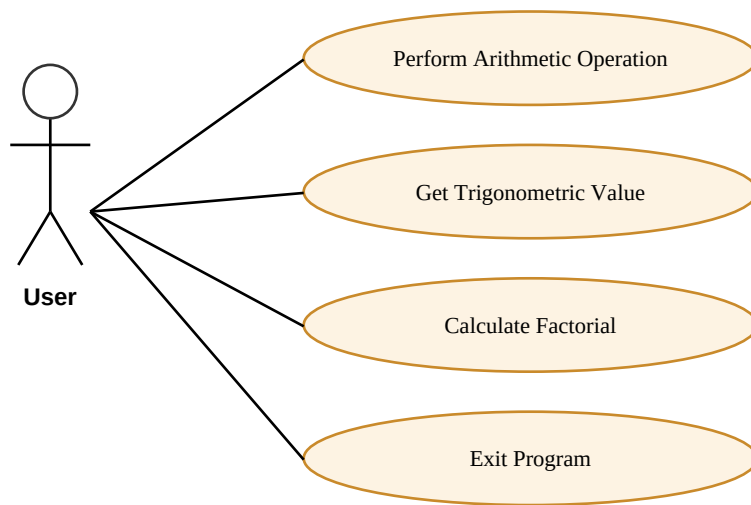


Figure 2: Use Case Diagram

6.2 Process Flow / Workflow Diagram

The flow below shows how the program moves from asking the initial question all the way to printing a result and looping back.

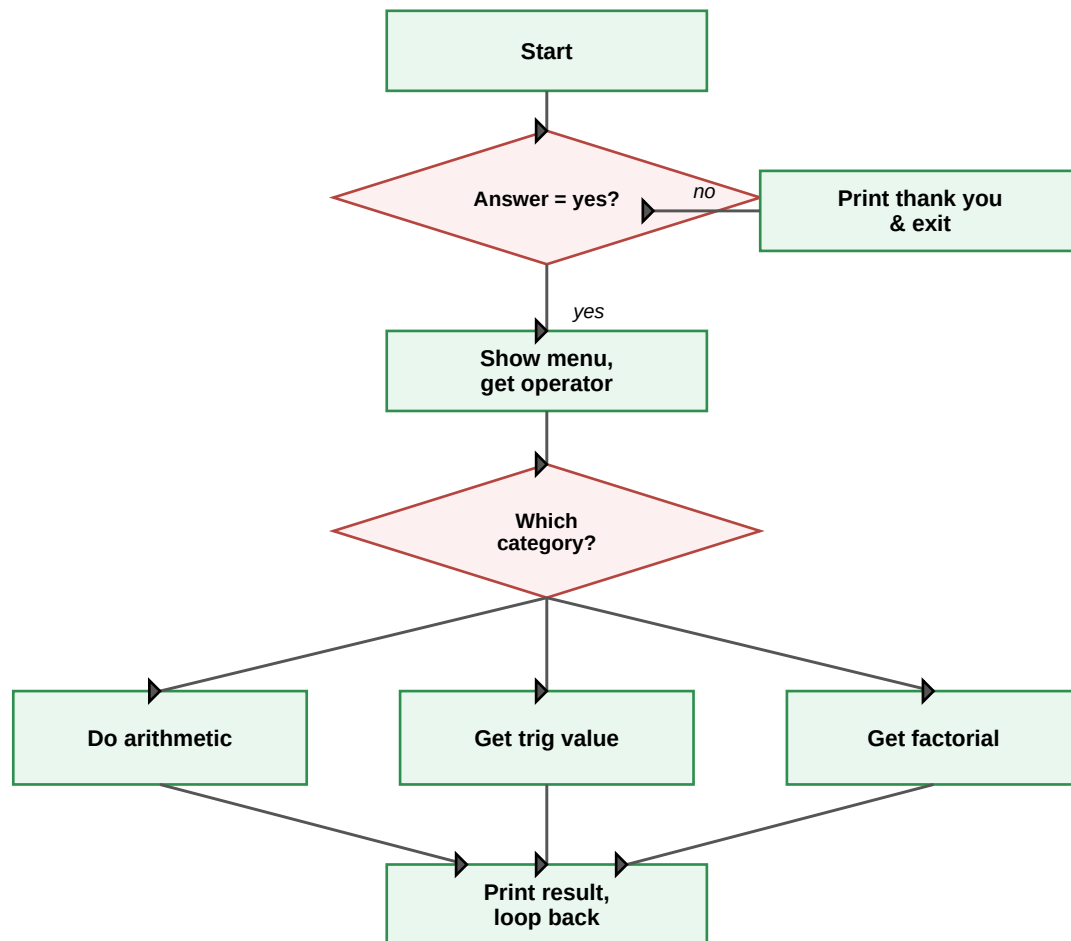


Figure 3: Process Flow Diagram (the 'no' path from the first decision leads to the thank-you/exit box; the last box loops back to Start)

6.3 Sequence Diagram

A simplified sequence diagram showing the message flow between the User and the Program for one calculation cycle.

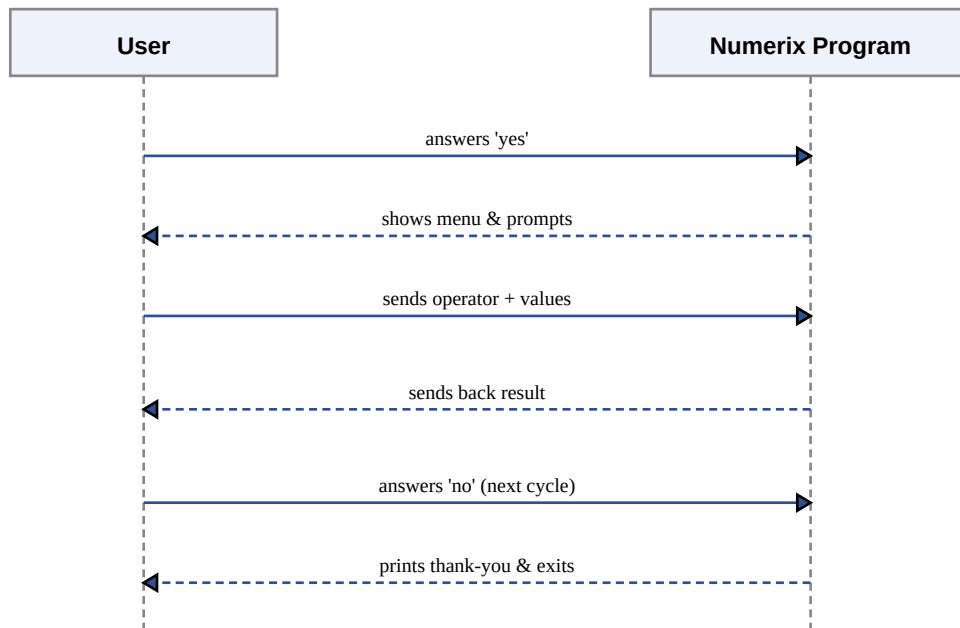


Figure 4: Sequence Diagram (one calculation cycle)

6.4 Component Diagram (in place of a Class Diagram)

Numerix is written in a **procedural** style rather than an object-oriented one — it doesn't define any classes, so a traditional class diagram doesn't really apply here. Instead, the diagram below shows the main components (functions and logic blocks) and how they relate to each other, which serves the same purpose of showing the program's structure.

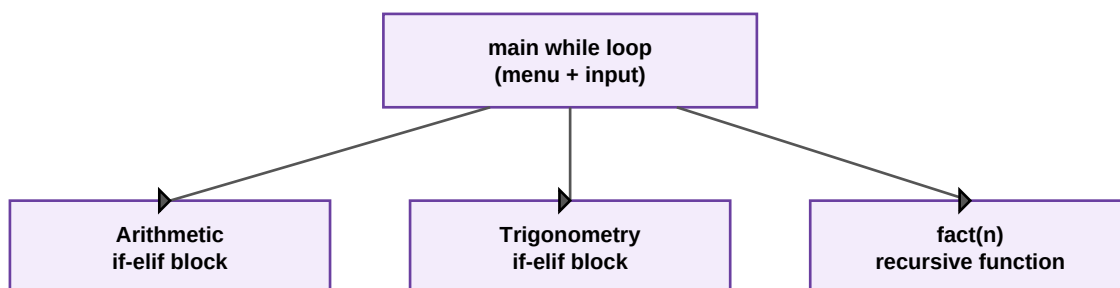


Figure 5: Component Diagram

6.5 ER Diagram / Database Design

Not applicable. Numerix doesn't use any database or persistent storage — every calculation is done in memory and nothing is saved once the program ends, so there was no schema or entity-relationship design needed for this project.

7. Design Decisions & Rationale

- **Using lists (ap, TP, FT) to validate the chosen operator** — instead of writing a long chain of comparisons upfront, I check if the user's input exists in a list first (using the `in` keyword) before going into the detailed if-elif logic. It made the top-level category check a lot shorter to write.

- **Recursion for factorial** — I used a recursive `fact()` function instead of a loop mainly because recursion was a topic we covered in class and I wanted to actually apply it somewhere instead of just doing the loop version.
- **Restricting trigonometry to five standard angles** — rather than importing Python's `math` module and computing `sin/cos/tan` for any angle, I hard-coded the known exact values for 0° , 30° , 45° , 60° and 90° . This was a deliberate scope decision to keep the project limited to core logic (if-elif chains) rather than relying on a library to do the actual math.
- **Menu-driven infinite loop with a yes/no check** — I went with a `while True` loop that only breaks when the user says anything other than 'yes', so the user can do several calculations in one run instead of restarting the program every time.
- **Single-file structure** — for a project this size, I kept everything in one Python file rather than splitting it into multiple files. In hindsight, this doesn't fully match the 5–10 file structure the project guidelines mention, and it's something I've flagged honestly under 'Challenges Faced' and 'Future Enhancements' rather than pretending the file is split up when it isn't.

8. Implementation Details

The core of the program is one file, `Numerix_Project.py`, structured roughly like this:

- Three lookup lists at the top: `ap` (allowed arithmetic operators), `TP` (allowed trig operators) and `FT` (the factorial operator, `C!`).
- A recursive `fact(n)` function that returns 1 for $n = 0$ or 1, and otherwise returns n multiplied by `fact(n-1)`.
- A banner printed with a simple for-loop that prints 70 '=' characters, used at the top and bottom of each run.
- The main `while True` loop, which asks the yes/no question, shows the menu, reads the operator, and then branches into whichever if-elif block matches (arithmetic / trig / factorial).
- Each branch reads the required inputs (one or two numbers, or an angle), does the calculation with plain Python operators, and prints the result.

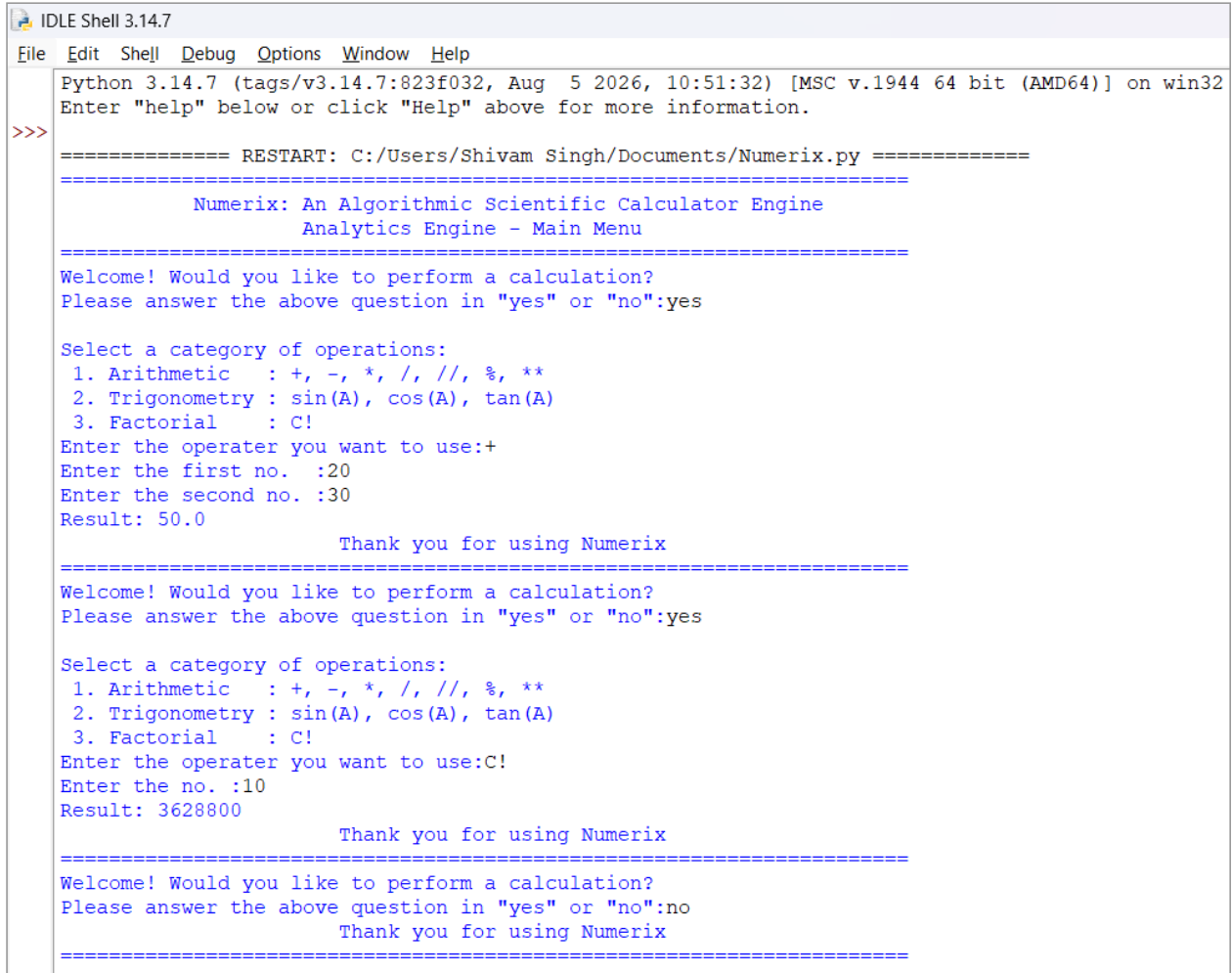
Sample Code Snippet

```
def fact(n):
    if n==0 or n==1:
        return 1
    else:
        return n * fact(n-1)
```

This is the recursive factorial function — it's a classic example of a base case (0 or 1) and a recursive case, which is exactly what we studied for recursion in class.

9. Screenshots / Results

Numerix is a console-based program, so its output is shown below as a real screenshot of the program running in Python IDLE. The run covers an arithmetic calculation ($20 + 30$), a factorial calculation ($10!$), and exiting the program by answering "no".



```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Shivam Singh/Documents/Numerix.py =====
=====
Numerix: An Algorithmic Scientific Calculator Engine
Analytics Engine - Main Menu
=====
Welcome! Would you like to perform a calculation?
Please answer the above question in "yes" or "no":yes

Select a category of operations:
1. Arithmetic : +, -, *, /, //, %, **
2. Trigonometry : sin(A), cos(A), tan(A)
3. Factorial : C!
Enter the operator you want to use:+
Enter the first no. :20
Enter the second no. :30
Result: 50.0

Thank you for using Numerix
=====
Welcome! Would you like to perform a calculation?
Please answer the above question in "yes" or "no":yes

Select a category of operations:
1. Arithmetic : +, -, *, /, //, %, **
2. Trigonometry : sin(A), cos(A), tan(A)
3. Factorial : C!
Enter the operator you want to use:C!
Enter the no. :10
Result: 3628800

Thank you for using Numerix
=====
Welcome! Would you like to perform a calculation?
Please answer the above question in "yes" or "no":no
Thank you for using Numerix
=====
```

Figure 6: Sample terminal run in IDLE showing addition ($20 + 30$), factorial ($10!$) and program exit

10. Testing Approach

Testing for this project was done **manually** by running the program repeatedly and trying a mix of normal and edge-case inputs. There's no automated test suite (like `unittest`) yet — that's listed as a future improvement. The table below lists the main test cases tried.

#	Test Case	Input	Expected Result	Actual Result
1	Simple addition	+, 10, 5	Result: 15.0	As expected
2	Division by zero	/, 10, 0	Undefined (division by zero).	As expected
3	Valid trig angle	sin(A), 30	Result:1/2	As expected
4	Invalid trig angle	sin(A), 20	"only allowed" message	As expected
5	Factorial of 5	C!, 5	Result: 120	As expected
6	Invalid operator	xyz	Invalid operator	As expected
7	Exit on first prompt	no	Thank-you message, exits	As expected

One thing testing did reveal: the exponent operator (`**`) is handled inside the if-elif block, but it isn't actually included in the `ap` list that's checked first — so trying to use it currently falls through to 'Invalid operator selected.' This is noted under Challenges Faced below rather than glossed over.

11. Challenges Faced

- **Division by zero** — had to remember to check for this specifically before dividing, otherwise Python throws an error and the whole program stops.
- **The exponent operator bug** — while testing, I found that `**` is handled in the if-elif chain but was left out of the `ap` validation list (there's an empty string in that list instead), so it never actually gets reached. I've left the bug in and documented it here rather than quietly fixing it after the fact, since it came up during testing.

- **Restricting trig values without a math library** — deciding to hard-code only 5 angles instead of using `math.sin()` etc. meant more manual if-elif writing, and I had to double check the exact fraction values for each angle by hand.
- **Negative numbers in factorial** — the `fact()` function doesn't check for negative input, so entering a negative number causes infinite recursion until Python hits its recursion limit and crashes. This wasn't caught until late testing.
- **Keeping the menu loop clean** — getting the while loop to correctly ask again after every calculation, and only exit on a non-'yes' answer, took a couple of tries to get the indentation and break logic right.
- **Single-file structure** — the project guidelines suggest splitting code into 5–10 files, but given the small scope of this calculator, everything ended up in one file. This is called out honestly rather than being hidden.

12. Learnings & Key Takeaways

- Got a much better feel for how while loops and if-elif chains work together to build an interactive, menu-driven program instead of a script that just runs once top to bottom.
- Understood recursion a lot better after actually writing and testing the `fact()` function myself, instead of just reading about it.
- Learned that input validation matters a lot more than I expected — almost every bug I found during testing was related to input I hadn't thought to handle (zero, negative numbers, unsupported angles, bad operators).
- Realised that keeping code in one file works for a small project, but even at this size it's already a bit harder to navigate than I expected, which made the case for modular file structure a lot more real to me.

13. Future Enhancements

- Fix the exponent operator (`**`) so it's properly included in the arithmetic validation list.
- Add a check for negative numbers in the `fact()` function so it doesn't cause infinite recursion.
- Split the single file into separate modules, e.g. `arithmetic.py`, `trigonometry.py`, `factorial.py` and `main.py`, to better match good project structure.
- Optionally allow trigonometric values for any angle using Python's `math` module, while keeping the current hard-coded mode as a 'no-library' option.
- Add basic automated tests using `unittest` instead of relying only on manual testing.
- Add a simple calculation history feature (stored in memory or a text file) so users can see what they calculated earlier in the same session.

14. References

- Python official documentation — docs.python.org
- Course lecture notes and slides on loops, functions and recursion

- General Python tutorials on recursion and control flow (GeeksforGeeks, W3Schools) used for concept revision